Trilha de SQL para Engenharia de Dados

Oct 15, 2026 · @Bruno Feliciano Martins

Como usar esta trilha

Todos os exercícios usam a mesma base real, a do e-commerce brasileiro Olist, no PostgreSQL. Comece pelo Módulo 0 (montar o banco), depois siga os módulos em ordem: assista às aulas do tópico e resolva os exercícios do mais fácil ao mais difícil, sem IA. São 16 módulos (do 0 ao 15) com 184 exercícios, mais 8 desafios integradores no final.

Níveis de dificuldade

Nível	O que significa	Meta
[F] Fácil	Aplica o comando direto	Até 10 min
[M] Médio	Combina dois conceitos ou exige cuidado com duplicidade e NULL	Até 25 min
[MD] Médio-difícil	Exige escolher a abordagem (CTE, janela, subconsulta) ou várias etapas	Até 45 min
[D] Difícil	Problema de negócio aberto ou de engenharia, como os de entrevista e do dia a dia	Até 1h30

Regra para avançar: resolva todos os Fáceis e Médios e pelo menos metade dos Médio-difíceis de um módulo antes de ir para o próximo.

Método para todo exercício

1. Diga em português o que cada linha do resultado representa (o "grão"): um pedido? um item? um cliente por mês?
2. Liste as tabelas e colunas necessárias e como elas se ligam.
3. Escreva a consulta em etapas, conferindo o resultado parcial a cada passo (de preferência com CTEs).
4. Valide: conte as linhas antes e depois de cada JOIN, procure duplicidades e NULL, e confira um caso à mão.
5. Releia a consulta e melhore a legibilidade: nomes de alias claros, uma cláusula por linha.

Regras de uso da IA

- Tente sozinho por 30 a 45 minutos antes de pedir ajuda.
- Peça dicas, nunca a consulta pronta: "qual cláusula ou função devo estudar para isso?".

- Depois de ver uma solução, refaça do zero no dia seguinte.
- Mande suas consultas para o Claude corrigir como tutor.

Base de dados: Olist (e-commerce brasileiro)

A base escolhida é o Brazilian E-Commerce Public Dataset by Olist, com cerca de 100 mil pedidos reais (anonimizados) feitos entre 2016 e 2018 em marketplaces brasileiros. Baixe o .zip no Kaggle (precisa de conta gratuita) e descompacte os 9 arquivos CSV em uma pasta.

Por que ela é boa para engenharia de dados: são 9 tabelas que se relacionam, com datas, valores, geografia, texto livre e problemas reais de qualidade, como duplicidades, NULL e erros de digitação. Dá para praticar desde SELECT simples até modelagem dimensional e cargas incrementais.

Tabelas (nome sugerido no banco, arquivo CSV e grão)

Tabela	Arquivo	Uma linha representa	Colunas principais
`orders`	olist_orders_dataset.csv	Um pedido	order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date
`order_items`	olist_order_items_dataset.csv	Um item de um pedido	order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value
`order_payments`	olist_order_payments_dataset.csv	Um pagamento de um pedido	order_id, payment_sequential, payment_type, payment_installments, payment_value
`order_reviews`	olist_order_reviews_dataset.csv	Uma avaliação	review_id, order_id, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp
`customers`	olist_customers_dataset.csv	Um cliente em um pedido	customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
`sellers`	olist_sellers_dataset.csv	Um vendedor	seller_id, seller_zip_code_prefix, seller_city, seller_state
`products`	olist_products_dataset.csv	Um produto	product_id, product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm

Tabela	Arquivo	Uma linha representa	Colunas principais
`category_translation`	product_category_name_translation.csv	Uma categoria	product_category_name, product_category_name_english
`geolocation`	olist_geolocation_dataset.csv	Um ponto de um CEP (com repetições)	geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state

Armadilhas conhecidas (parte do aprendizado)

- `customer_id` muda a cada pedido. Para saber quem é a mesma pessoa, use `customer_unique_id`.
- Um pedido pode ter vários itens e vários pagamentos. Juntar `order_items` com `order_payments` direto multiplica linhas.
- `geolocation` tem vários pontos por CEP. Juntar sem agregar antes explode o resultado.
- Os nomes `product_name_lenght` e `product_description_lenght` vêm com erro de digitação no arquivo original.
- Há pedidos sem data de entrega, produtos sem categoria e avaliações com `review_id` repetido.
- Os CEPs têm só os 5 primeiros dígitos e as cidades aparecem com grafias diferentes (acentos, maiúsculas).

Verifique os nomes exatos das colunas no cabeçalho de cada CSV antes de criar as tabelas.

Ambiente e Módulo 0: montar o banco

Use o PostgreSQL (versão 15 ou mais recente, por causa do comando `MERGE`) e uma ferramenta para escrever consultas, como o DBeaver ou o pgAdmin, que já vem com o instalador do PostgreSQL. Se preferir, rode o PostgreSQL em Docker.

Montar o banco já é o primeiro conjunto de exercícios: carregar arquivos brutos em tabelas é uma tarefa diária de engenharia de dados.

1. [F] Instale o PostgreSQL e crie um banco chamado `olist`.
2. [F] Crie um schema `raw` para os dados brutos. Por que separar dados brutos dos dados tratados?
3. [F] Abra cada CSV num editor de texto e anote: separador, se tem cabeçalho, formato das datas e quais colunas parecem números, datas ou texto.
4. [M] Crie as 9 tabelas no schema `raw` com `CREATE TABLE`, escolhendo um tipo para cada coluna (`TEXT`, `INTEGER`, `NUMERIC(10,2)`, `TIMESTAMP`, `DOUBLE PRECISION`).

5. [M] Carregue os CSVs com o comando `\copy` do psql (ou `COPY`, ou a importação do DBeaver). Anote os erros que aparecerem e corrija.
6. [M] Confira a carga: conte as linhas de cada tabela com `COUNT(*)` e compare com o número de linhas de cada CSV (menos o cabeçalho).
7. [M] Tente criar chave primária em `raw.order_reviews(review_id)`. Por que falha? O que isso diz sobre os dados?
8. [MD] Escreva um script `.sql` único que apaga e recria todas as tabelas e recarrega os dados do zero. Rode duas vezes seguidas: o resultado tem que ser o mesmo (idempotência).
9. [MD] Alternativa de carga: carregue tudo como `TEXT` primeiro e depois converta os tipos com `INSERT INTO ... SELECT` com `CAST`. Compare com a carga tipada direta: qual é mais robusta para dados sujos?
10. [D] Escreva uma consulta que mostre, para cada tabela do schema `raw`, o número de linhas e o tamanho em disco, consultando o catálogo do PostgreSQL (`information_schema` e `pg_total_relation_size`).

Roteiro de estudos

Use os termos abaixo como busca de videoaula (ex.: "postgresql window functions"). Cada módulo corresponde a uma seção de exercícios.

Módulo	Tópicos para estudar
0. Ambiente e carga	Instalação do PostgreSQL; psql e DBeaver; banco e schema; CREATE TABLE; tipos de dados; COPY e \copy
1. Consultas básicas	SELECT e alias; WHERE; operadores AND, OR, NOT; IN, BETWEEN, LIKE e ILIKE; IS NULL; ORDER BY; LIMIT e OFFSET; DISTINCT; ordem lógica de execução de uma consulta
2. Funções e CASE	Funções de texto (LOWER, UPPER, TRIM, SUBSTRING, LENGTH, REPLACE, CONCAT); funções numéricas (ROUND, CEIL, FLOOR); CAST e `::`; COALESCE e NULLIF; CASE WHEN; datas básicas (EXTRACT, DATE_TRUNC, AGE)
3. Agregação	COUNT, SUM, AVG, MIN, MAX; COUNT(DISTINCT); GROUP BY; HAVING; FILTER (WHERE ...); agregação condicional com CASE; WHERE vs. HAVING

Módulo	Tópicos para estudar
4. JOINs	INNER, LEFT, RIGHT e FULL JOIN; CROSS JOIN; self join; anti-join (LEFT JOIN ... IS NULL); grão das tabelas; multiplicação de linhas (fan-out)
5. Subconsultas	Subconsulta escalar; IN e NOT IN; EXISTS e NOT EXISTS; subconsulta correlacionada; subconsulta no FROM; armadilha do NOT IN com NULL
6. CTEs	WITH; várias CTEs encadeadas; CTE vs. subconsulta; legibilidade e depuração em etapas
7. Funções de janela	OVER, PARTITION BY, ORDER BY; ROW_NUMBER, RANK, DENSE_RANK; LAG e LEAD; SUM e AVG acumulados; frame ROWS BETWEEN; NTILE; FIRST_VALUE e LAST_VALUE; QUALIFY (alternativa com subconsulta no PostgreSQL)
8. NULL e qualidade de dados	Lógica de três valores; NULL em agregações e JOINs; deduplicação com ROW_NUMBER; testes de unicidade, completude e integridade referencial; padronização de texto
9. Datas e séries temporais	TIMESTAMP e INTERVAL; DATE_TRUNC; generate_series; preencher dias sem dados; médias móveis; crescimento mês a mês; análise de coorte
10. DDL, constraints e transações	CREATE, ALTER e DROP; PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK; normalização (1FN, 2FN, 3FN); BEGIN, COMMIT e ROLLBACK; propriedades ACID
11. Modelagem dimensional	Tabelas fato e dimensão; esquema estrela; grão da fato; chaves substitutas (surrogate keys); dimensão de datas; métricas aditivas e não aditivas
12. Cargas incrementais e SCD	INSERT ... ON CONFLICT (upsert); MERGE; carga incremental por data de atualização; idempotência; SCD tipo 1 e tipo 2; camadas bronze, silver e gold
13. Performance	EXPLAIN e EXPLAIN ANALYZE; Seq Scan vs. Index Scan; índices B-tree e compostos; índice parcial; custo de JOIN; particionamento de tabelas (introdução)

Módulo	Tópicos para estudar
14. Views, conjuntos e recursão	VIEW e MATERIALIZED VIEW; UNION, UNION ALL, INTERSECT, EXCEPT; CTE recursiva; funções em SQL (introdução)
15. Lógica avançada em SQL	Gaps and islands; sessões e sequências; mediana e percentis (PERCENTILE_CONT); pivot com agregação condicional; problemas estilo entrevista

Módulo 1: Consultas básicas

1. [F] Mostre as 10 primeiras linhas de `orders`.
2. [F] Liste os pedidos com status `delivered`, mostrando só `order_id`, `customer_id` e `order_purchase_timestamp`.
3. [F] Liste os valores distintos de `order_status`.
4. [F] Liste os valores distintos de `payment_type`.
5. [F] Mostre os 20 itens mais caros de `order_items`, do maior para o menor preço.
6. [F] Liste os clientes do estado de São Paulo (`SP`) da cidade de Campinas.
7. [F] Liste os vendedores dos estados do Sul (PR, SC e RS) usando `IN`.
8. [M] Liste os pedidos feitos em dezembro de 2017 usando `BETWEEN` ou comparações de data. Cuidado com o último dia e o horário.
9. [M] Liste os pedidos que ainda não têm data de entrega ao cliente.
10. [M] Liste as categorias de produto que contêm a palavra "casa" no nome, sem diferenciar maiúsculas de minúsculas.
11. [M] Liste os pagamentos parcelados em mais de 10 vezes no cartão de crédito, ordenados pelo valor.
12. [M] Mostre a página 3 de uma listagem de produtos com 25 produtos por página, ordenada por `product_id`.
13. [M] Liste os itens cujo frete (`freight_value`) é maior que o próprio preço do produto.
14. [MD] Explique por que esta consulta dá erro e corrija: `SELECT price * 2 AS dobro FROM order_items WHERE dobro > 100;`. Escreva em uma frase a ordem lógica em que o banco executa SELECT, FROM, WHERE e ORDER BY.
15. [MD] Liste as avaliações com nota 1 que têm comentário escrito, ordenadas da mais recente para a mais antiga.

Módulo 2: Funções e CASE

1. [F] Mostre o nome da cidade dos clientes em maiúsculas e o tamanho do nome em caracteres.
2. [F] Mostre o preço de cada item arredondado para inteiro e o preço mais frete com 2 casas decimais.
3. [F] Mostre ano, mês e dia da semana da compra de cada pedido usando `EXTRACT`.
4. [F] Mostre a categoria do produto ou o texto "sem_categoria" quando ela for NULL, usando `COALESCE`.
5. [M] Calcule, para cada pedido entregue, quantos dias levou entre a compra e a entrega ao cliente.
6. [M] Classifique cada pedido entregue em "no prazo" ou "atrasado", comparando a data de entrega com a data estimada.
7. [M] Crie uma faixa de preço para cada item: até 50, de 50 a 200, de 200 a 1000 e acima de 1000.
8. [M] Traduza a nota da avaliação em texto: 1 e 2 "negativa", 3 "neutra", 4 e 5 "positiva".
9. [M] Mostre o CEP do cliente sempre com 5 dígitos, completando com zeros à esquerda (o CSV pode ter perdido os zeros).
10. [M] Calcule o volume de cada produto em litros (comprimento × altura × largura em cm, dividido por 1000) e trate produtos sem medidas.
11. [MD] Padronize os nomes de cidade dos vendedores: minúsculas, sem espaços nas pontas e sem acentos (pesquise a extensão `unaccent`).
12. [MD] Calcule a razão frete sobre preço de cada item usando `NULLIF` para evitar divisão por zero.
13. [MD] Extraia o primeiro nome da categoria (até o primeiro "_") de `product_category_name` (ex.: "cama_mesa_banho" vira "cama").
14. [D] Crie uma coluna "turno da compra" (madrugada, manhã, tarde, noite) a partir da hora da compra e outra "tipo de dia" (dia útil ou fim de semana).

Módulo 3: Agregação

1. [F] Conte quantos pedidos existem.
2. [F] Conte quantos pedidos existem por status.
3. [F] Calcule o preço médio, mínimo e máximo dos itens.
4. [F] Conte quantos clientes únicos (`customer_unique_id`) existem.
5. [F] Conte quantos vendedores existem por estado, do maior para o menor.
6. [M] Calcule o faturamento total (preço + frete) por ano e mês de compra.

7. [M] Liste os estados com mais de 1.000 clientes únicos usando `HAVING`.
8. [M] Calcule a nota média das avaliações e a quantidade de avaliações por nota.
9. [M] Para cada tipo de pagamento, calcule a quantidade de pagamentos, o valor total e a média de parcelas.
10. [M] Em uma única consulta, conte os pedidos entregues, cancelados e os demais, usando `COUNT(*) FILTER (WHERE ...)`.
11. [M] Calcule quantos itens tem cada pedido e depois mostre a distribuição: quantos pedidos têm 1 item, 2 itens e assim por diante.
12. [MD] Calcule o percentual de pedidos entregues com atraso em relação ao total de pedidos entregues.
13. [MD] Liste os 10 vendedores com maior faturamento que venderam pelo menos 50 itens.
14. [MD] Calcule o ticket médio por pedido. Por que `AVG(price)` em `order_items` não é o ticket médio?
15. [D] Monte uma tabela com uma linha por estado do cliente e uma coluna por tipo de pagamento, com o valor total pago (pivot com agregação condicional).

Módulo 4: JOINs

Antes de cada JOIN, pergunte: qual é o grão de cada tabela? A chave de ligação se repete em algum dos lados?

1. [F] Liste os pedidos com o estado e a cidade do cliente.
2. [F] Liste os itens vendidos com a categoria do produto em português e em inglês.
3. [F] Liste os itens vendidos com o estado do vendedor.
4. [M] Calcule o faturamento por categoria em inglês, do maior para o menor.
5. [M] Calcule o faturamento por estado do cliente.
6. [M] Encontre os produtos que nunca foram vendidos (anti-join com `LEFT JOIN ... IS NULL`).
7. [M] Encontre os pedidos que não têm avaliação.
8. [M] Encontre as categorias de `products` que não aparecem em `category_translation`.
9. [M] Liste os pedidos em que cliente e vendedor são do mesmo estado. Que proporção dos pedidos isso representa?
10. [MD] Conte as linhas de `orders`, depois de `orders` com `order_items`, depois com `order_payments` também. Explique por que o número cresce e por que somar `payment_value` nesse resultado dá um valor errado.

11. [MD] Calcule corretamente, por pedido, o total dos itens (preço + frete) e o total pago, agregando cada tabela antes do JOIN. Liste os pedidos em que os dois valores diferem em mais de 1 real.
12. [MD] Calcule a nota média de avaliação por categoria de produto, considerando só categorias com pelo menos 100 avaliações.
13. [MD] Use um self join em `order_items` para encontrar pares de produtos diferentes comprados no mesmo pedido e liste os 10 pares mais frequentes.
14. [D] Calcule a distância média entre cliente e vendedor por pedido usando `geolocation` (latitude e longitude). Antes, reduza `geolocation` a um ponto por CEP para não multiplicar linhas.
15. [D] Use `FULL JOIN` para comparar a lista de estados dos clientes com a dos vendedores, mostrando os estados que só têm clientes, só têm vendedores e os que têm os dois.

Módulo 5: Subconsultas

1. [F] Liste os itens com preço acima do preço médio de todos os itens (subconsulta escalar).
2. [F] Liste os pedidos dos clientes do Rio de Janeiro usando `IN` com subconsulta, sem JOIN.
3. [M] Liste os vendedores que venderam algum produto da categoria "beleza_saude", usando `EXISTS`.
4. [M] Liste os clientes únicos que nunca deram nota menor que 4, usando `NOT EXISTS`.
5. [M] Liste os itens cujo preço é maior que a média de preço da sua própria categoria (subconsulta correlacionada).
6. [M] Mostre cada categoria com seu faturamento e o percentual que ela representa do faturamento total, usando uma subconsulta escalar no SELECT.
7. [MD] Compare `NOT IN` e `NOT EXISTS` para achar produtos nunca vendidos. Depois insira de propósito uma linha com `product_id` NULL numa tabela de teste e explique por que o `NOT IN` passa a retornar zero linhas.
8. [MD] Encontre o pedido mais caro de cada estado do cliente usando subconsulta correlacionada. Depois refaça com função de janela no Módulo 7 e compare.
9. [MD] Liste os vendedores cujo faturamento está acima da média de faturamento dos vendedores do mesmo estado.
10. [D] Encontre os clientes únicos que compraram em pelo menos 3 meses diferentes, usando apenas subconsultas no FROM.

Módulo 6: CTEs

A partir daqui, prefira CTEs: elas deixam cada etapa testável separadamente.

1. [F] Reescreva o exercício 6 do Módulo 5 usando uma CTE.
2. [M] Com uma CTE de totais por pedido, calcule o ticket médio por estado do cliente.
3. [M] Com duas CTEs (faturamento por vendedor e quantidade de pedidos por vendedor), monte um relatório com faturamento, pedidos e ticket médio por vendedor.
4. [M] Calcule a nota média de cada vendedor (avaliação ligada ao pedido, pedido ligado aos itens) e liste os 10 piores com pelo menos 30 avaliações.
5. [MD] Monte uma tabela "pedido enriquecido" em CTEs encadeadas: dados do pedido, cliente, total de itens, total pago, nota da avaliação, dias de entrega e flag de atraso. Uma linha por pedido.
6. [MD] Usando o resultado anterior, compare a nota média de pedidos atrasados e pedidos no prazo.
7. [MD] Calcule quantos pedidos cada cliente único fez e mostre a distribuição: quantos clientes fizeram 1 pedido, 2 pedidos, 3 ou mais. Qual é a taxa de recompra?
8. [D] Calcule, para cada categoria, a participação dos 3 maiores vendedores no faturamento da categoria (concentração de mercado).

Módulo 7: Funções de janela

1. [F] Numere os itens de cada pedido com `ROW_NUMBER()` ordenando pelo preço do maior para o menor.
2. [F] Mostre cada item com o preço médio da sua categoria ao lado, sem agrupar (`AVG() OVER`).
3. [M] Faça um ranking dos vendedores por faturamento dentro de cada estado e mostre os 3 primeiros de cada estado.
4. [M] Compare `ROW_NUMBER`, `RANK` e `DENSE_RANK` num ranking de categorias por quantidade de itens vendidos. Quando o resultado muda?
5. [M] Para cada cliente único, numere seus pedidos em ordem de compra e identifique o primeiro pedido.
6. [M] Calcule o faturamento por mês e o faturamento acumulado ao longo do tempo.
7. [M] Calcule o faturamento por mês e a variação percentual em relação ao mês anterior com `LAG`.
8. [MD] Para cada cliente único com mais de um pedido, calcule quantos dias se passaram entre cada compra e a anterior. Qual é o intervalo médio de recompra?
9. [MD] Calcule a média móvel de 7 dias do número de pedidos por dia, usando `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.

10. [MD] Divida os clientes únicos em 4 grupos pelo valor total gasto com `NTILE(4)` e mostre o gasto mínimo, máximo e médio de cada grupo.
11. [MD] Para cada categoria, mostre o produto mais vendido e o segundo mais vendido lado a lado.
12. [MD] Calcule o percentual de cada vendedor no faturamento do seu estado, usando `SUM() OVER (PARTITION BY ...)`.
13. [D] Curva ABC de produtos: ordene os produtos por faturamento, calcule o percentual acumulado e classifique em A (até 80%), B (até 95%) e C (o resto).
14. [D] Para cada pedido entregue, compare o tempo de entrega com a média dos 30 pedidos anteriores do mesmo estado do cliente.

Módulo 8: NULL e qualidade de dados

Esse é o dia a dia de engenharia de dados: antes de confiar num dado, teste.

1. [F] Conte quantos valores NULL há em cada coluna de `orders`.
2. [F] Compare `COUNT(*)`, `COUNT(order_delivered_customer_date)` e `COUNT(DISTINCT customer_id)` em `orders` e explique cada número.
3. [F] Verifique se `order_id` é único em `orders` (o teste: o número de linhas é igual ao número de valores distintos?).
4. [M] Encontre os `review_id` repetidos em `order_reviews` e mostre as linhas completas de alguns deles. São duplicatas exatas ou não?
5. [M] Teste a integridade referencial: existem itens cujo `product_id` não está em `products`? Pedidos cujo `customer_id` não está em `customers`? Itens cujo pedido não existe?
6. [M] Encontre pedidos com datas incoerentes: aprovação antes da compra, entrega antes do envio à transportadora ou entrega antes da compra.
7. [M] Encontre pedidos com status `delivered` sem data de entrega e pedidos com data de entrega mas status diferente de `delivered`.
8. [M] Liste os pagamentos com `payment_type` igual a `not_defined` ou valor zero. O que eles significam?
9. [MD] Deduplique `order_reviews` mantendo, para cada `review_id`, só a avaliação respondida mais recentemente (`ROW_NUMBER` + filtro).
10. [MD] Quantas grafias diferentes existem para a mesma cidade em `geolocation`? Liste as cidades com mais variações (ex.: com e sem acento).
11. [MD] Monte uma consulta de "relatório de qualidade" que retorna uma linha por teste (nome do teste, tabela, quantidade de linhas com problema), juntando vários testes com `UNION ALL`.

12. [D] Crie o schema `silver` com versões limpas das tabelas: tipos corretos, textos padronizados, avaliações deduplicadas, uma linha por CEP em geolocation e categoria traduzida nos produtos. Documente cada regra de limpeza com um comentário.

Módulo 9: Datas e séries temporais

1. [F] Mostre a data da primeira e da última compra da base.
2. [F] Conte os pedidos por dia da semana. Qual é o dia com mais compras?
3. [F] Conte os pedidos por hora do dia.
4. [M] Conte os pedidos por semana usando `DATE_TRUNC('week', ...)`.
5. [M] Gere com `generate_series` uma lista com todos os dias entre a primeira e a última compra e junte com a contagem de pedidos por dia, mostrando 0 nos dias sem pedido.
6. [M] Liste os dias em que não houve nenhum pedido.
7. [M] Calcule o tempo médio, em horas, entre a compra e a aprovação do pagamento, por tipo de pagamento.
8. [MD] Calcule o crescimento de faturamento de cada mês em relação ao mesmo mês do ano anterior.
9. [MD] Compare o faturamento da Black Friday de 2017 (24/11) com a média das 4 sextas-feiras anteriores.
10. [MD] Calcule o atraso médio (data entregue menos data estimada) por mês de compra e por estado do cliente.
11. [D] Análise de coorte: agrupe os clientes únicos pelo mês da primeira compra e mostre, para cada coorte, quantos voltaram a comprar 1, 2, 3 até 6 meses depois.
12. [D] Calcule, para cada mês, quantos clientes estavam "ativos" (compraram nos últimos 90 dias até o fim daquele mês).

Módulo 10: DDL, constraints e transações

1. [F] Crie o schema `silver` (se ainda não criou) e uma tabela `silver.orders` com os tipos corretos.
2. [F] Adicione chave primária em `silver.orders(order_id)`.
3. [M] Crie `silver.order_items` com chave primária composta (`order_id`, `order_item_id`) e chave estrangeira para `silver.orders`.
4. [M] Adicione constraints `CHECK` em `silver.order_items`: preço maior que zero e frete maior ou igual a zero. Tente inserir uma linha inválida e veja o erro.
5. [M] Adicione uma `CHECK` em `silver.order_reviews` garantindo nota entre 1 e 5.

6. [M] Crie as chaves estrangeiras de todo o schema `silver`. Em que ordem as tabelas precisam ser carregadas para não dar erro?
7. [M] Dentro de uma transação, apague todos os pedidos cancelados de uma tabela de teste, confira a contagem e desfaça com `ROLLBACK`.
8. [MD] Avalie se `customers` está normalizada: a cidade e o estado dependem só do cliente ou do CEP? Proponha um modelo em 3FN com uma tabela de CEPs.
9. [MD] Adicione uma coluna `loaded_at TIMESTAMP DEFAULT now()` em todas as tabelas `silver`. Para que ela serve num pipeline?
10. [D] Escreva o script completo de criação do schema `silver` (tabelas, chaves, constraints e índices nas chaves estrangeiras) e carregue os dados a partir do `raw`, tratando as linhas que violarem as regras numa tabela de rejeitados.

Módulo 11: Modelagem dimensional

O objetivo é montar um schema `gold` em esquema estrela, pensado para análise e dashboards.

1. [F] Defina, em uma frase, o grão de uma tabela fato de vendas feita a partir de `order_items`.
2. [M] Crie `gold.dim_date` com uma linha por dia de 2016 a 2018: data, ano, mês, nome do mês, trimestre, dia da semana e flag de fim de semana (use `generate_series`).
3. [M] Crie `gold.dim_product` com chave substituta (`product_key` gerada pelo banco), `product_id`, categoria em português e inglês e medidas.
4. [M] Crie `gold.dim_customer` com uma linha por `customer_unique_id` (não por `customer_id`). Que dados usar quando o mesmo cliente aparece com CEPs diferentes?
5. [M] Crie `gold.dim_seller` com chave substituta.
6. [MD] Crie `gold.fact_sales` com uma linha por item vendido: chaves das dimensões, chave da data de compra, preço, frete e status do pedido.
7. [MD] Crie `gold.fact_orders` com uma linha por pedido: chaves de cliente e data, quantidade de itens, valor total, valor pago, dias de entrega, flag de atraso e nota da avaliação.
8. [MD] Usando só o schema `gold`, responda: faturamento por categoria em inglês e trimestre. Compare a consulta com a mesma pergunta feita nas tabelas `raw`.
9. [MD] Quais métricas de `fact_orders` podem ser somadas por qualquer dimensão e quais não (ex.: nota média, ticket médio)? Explique aditivas, semiaditivas e não aditivas.

10. [D] Desenhe (papel ou ferramenta como o dbdiagram.io) o diagrama do seu esquema estrela e escreva um dicionário de dados: descrição, grão e origem de cada tabela e coluna.

Módulo 12: Cargas incrementais e SCD

Para simular dados chegando ao longo do tempo, divida os pedidos por mês de compra e carregue um mês por vez.

1. [F] Crie uma tabela `staging.orders_batch` com o mesmo formato de `silver.orders` para receber cada lote.
2. [M] Carregue em `staging.orders_batch` só os pedidos de janeiro de 2017 e insira em `silver.orders` só os que ainda não existem (anti-join).
3. [M] Refaça a carga com `INSERT ... ON CONFLICT (order_id) DO NOTHING`. Rode duas vezes e confirme que não duplica.
4. [M] Simule uma atualização: altere o status de alguns pedidos no lote e use `ON CONFLICT ... DO UPDATE` para atualizar só as linhas que mudaram.
5. [MD] Faça a mesma carga com `MERGE`, tratando inserção, atualização e (opcional) exclusão em um único comando.
6. [MD] Crie uma tabela de controle `etl.watermark` que guarda a maior data carregada por tabela. Escreva a carga incremental que lê só pedidos com data de compra maior que a marca d'água e depois a atualiza.
7. [MD] Escreva a carga incremental de `gold.fact_sales` para um mês: apague o mês e reinsira (estratégia "delete + insert" por partição). Por que ela é idempotente?
8. [MD] Implemente SCD tipo 1 em `gold.dim_seller`: quando a cidade de um vendedor muda, sobrescreva.
9. [D] Implemente SCD tipo 2 em `gold.dim_seller`: quando a cidade muda, feche a versão antiga (`valid_to`, `is_current = false`) e insira uma nova versão. Simule duas mudanças para o mesmo vendedor e consulte a cidade dele em datas diferentes.
10. [D] Junte tudo num processo de 3 camadas: `raw` (bruto), `silver` (limpo e tipado) e `gold` (esquema estrela). Escreva um script por camada e rode a carga de 3 meses seguidos, de forma incremental.

Módulo 13: Performance

1. [F] Rode `EXPLAIN` numa consulta que busca um pedido pelo `order_id` em `raw.orders` (sem índice) e em `silver.orders` (com chave primária). Compare os planos.
2. [F] Rode `EXPLAIN ANALYZE` na consulta de faturamento por mês e anote o tempo total.

3. [M] Crie um índice em `order_items(product_id)` e compare o plano de uma consulta que filtra um produto antes e depois.
4. [M] Crie um índice em `orders(order_purchase_timestamp)` e teste uma consulta por intervalo de datas. O índice é usado para um intervalo de 1 dia? E para 2 anos? Por quê?
5. [M] Mostre por que `WHERE EXTRACT(YEAR FROM order_purchase_timestamp) = 2017` não usa o índice e reescreva a condição para que use.
6. [MD] Crie um índice composto em `orders(customer_id, order_purchase_timestamp)`. Em que consultas ele ajuda e em quais a ordem das colunas faz diferença?
7. [MD] Crie um índice parcial só para pedidos não entregues e explique quando esse tipo de índice compensa.
8. [MD] Compare o desempenho de `COUNT(DISTINCT ...)` com `GROUP BY` seguido de `COUNT(*)` numa tabela grande (use `geolocation`).
9. [D] Particione uma cópia de `fact_sales` por mês de compra (particionamento declarativo do PostgreSQL) e mostre no `EXPLAIN` que uma consulta de um mês lê só uma partição.

Módulo 14: Views, conjuntos e recursão

1. [F] Crie a view `vw_pedidos_entregues` com os pedidos entregues e seus dias de entrega.
2. [F] Liste os estados que têm clientes e também vendedores, usando `INTERSECT`.
3. [F] Liste os estados que têm clientes mas nenhum vendedor, usando `EXCEPT`.
4. [M] Junte as cidades de clientes e vendedores numa única lista com `UNION` e depois com `UNION ALL`. Qual a diferença na contagem e no desempenho?
5. [M] Crie a view materializada `mv_faturamento_mensal` e atualize com `REFRESH MATERIALIZED VIEW`. Quando usar view comum e quando materializada?
6. [M] Crie uma view de qualidade de dados (do Módulo 8) que qualquer pessoa pode consultar para ver se a última carga está saudável.
7. [MD] Use uma CTE recursiva para gerar os números de 1 a 100 e depois para gerar todos os meses entre a primeira e a última compra (sem `generate_series`).
8. [MD] Crie uma pequena tabela de categorias hierárquicas (ex.: "casa" > "cama_mesa_banho", "casa" > "moveis_decoracao") e use CTE recursiva para mostrar o caminho completo de cada categoria e o faturamento somado por nível.
9. [D] Crie uma função SQL `faturamento_periodo(inicio DATE, fim DATE)` que retorna o faturamento por categoria naquele período, e use-a numa consulta.

Módulo 15: Lógica avançada em SQL

Problemas que costumam cair em entrevista de engenharia e análise de dados.

1. [M] Encontre o segundo maior faturamento mensal sem usar `LIMIT` ou `OFFSET`.
2. [M] Calcule a mediana do preço dos itens e do tempo de entrega com `PERCENTILE_CONT(0.5)`. Compare com a média: o que a diferença indica?
3. [M] Liste os clientes únicos que compraram em dois meses consecutivos.
4. [MD] Gaps and islands: para cada vendedor, encontre as sequências de dias consecutivos com pelo menos uma venda e mostre a maior sequência de cada um.
5. [MD] Encontre, para cada mês, o vendedor com maior faturamento. Em caso de empate, mostre todos.
6. [MD] Calcule a taxa de retenção mês a mês: dos clientes que compraram no mês M, quantos compraram de novo no mês M+1?
7. [MD] Segmentação RFM: calcule, para cada cliente único, recência (dias desde a última compra até a última data da base), frequência (número de pedidos) e valor (total gasto), dê nota de 1 a 5 em cada um com `NTILE(5)` e classifique os segmentos.
8. [MD] Funil de pedidos: conte quantos pedidos passaram por cada etapa (comprado, aprovado, enviado à transportadora, entregue) e a taxa de conversão entre etapas.
9. [D] Market basket: calcule, para os pares de categorias compradas juntas, o suporte (frequência do par) e a confiança (de quem comprou A, quantos também compraram B).
10. [D] Encontre os vendedores cuja nota média caiu em 3 meses seguidos.
11. [D] Detecção de anomalias: encontre os dias em que o número de pedidos ficou mais de 3 desvios padrão acima ou abaixo da média móvel de 28 dias.

Desafios integradores

Mini-projetos de engenharia de dados. Cada um vira um bom item de portfólio no GitHub: guarde os scripts `.sql`, um README explicando as decisões e, se possível, um diagrama.

1. [MD] Dashboard de vendas: monte as consultas por trás de um painel com faturamento por mês, ticket médio, pedidos por estado, top 10 categorias e taxa de atraso, todas lendo do schema `gold` (Módulos 3, 7 e 11).
2. [MD] Relatório de logística: por estado do cliente e mês, mostre prazo médio, atraso médio, percentual de atrasos e nota média, e identifique as rotas (estado do vendedor para estado do cliente) mais problemáticas (Módulos 4, 6 e 9).
3. [MD] Scorecard de vendedores: para cada vendedor, faturamento, pedidos, nota média, percentual de atraso por culpa do envio (`shipping_limit_date` estourada) e ranking dentro do estado (Módulos 6 e 7).

4. [D] Pipeline completo em camadas: scripts idempotentes para `raw`, `silver` e `gold`, carga incremental por mês com marca d'água, tabela de log de execução (início, fim, linhas inseridas e atualizadas) e testes de qualidade que interrompem a carga se falharem (Módulos 0, 8, 10, 11 e 12).
5. [D] Framework de qualidade de dados: uma tabela de regras (nome, tabela, consulta SQL que retorna as linhas com problema, severidade) e um script que executa todas as regras e grava o resultado num histórico. Pesquise como fazer isso com SQL dinâmico (`EXECUTE`) numa função PL/pgSQL (Módulos 8 e 14).
6. [D] Análise de clientes: coorte de retenção, segmentação RFM, tempo médio até a segunda compra e valor do cliente nos primeiros 6 meses, entregues como views no schema `gold` (Módulos 9 e 15).
7. [D] Otimização: escolha as 5 consultas mais lentas dos desafios anteriores, meça com `EXPLAIN ANALYZE`, proponha índices, reescritas ou views materializadas, e documente o antes e o depois (Módulo 13).
8. [D] Reproduza o pipeline no DuckDB, lendo os CSVs direto com `read_csv`, e compare a experiência e o desempenho com o PostgreSQL. É uma ferramenta cada vez mais usada em engenharia de dados.

Fontes externas

Quando terminar um módulo, continue praticando nestas plataformas com o mesmo tema.

Fonte	Para que usar	Módulos
PostgreSQL Exercises	Exercícios gratuitos feitos para PostgreSQL, de consultas básicas a CTE recursiva, com correção no navegador	1 a 7, 14
SQLBolt	Lições curtas e interativas para quem está começando	1 a 4
LeetCode SQL 50	Plano com 50 problemas de SQL em ordem de dificuldade, no estilo de entrevistas	3 a 7, 15
DataLemur	Perguntas de SQL de entrevistas reais de empresas de tecnologia, várias gratuitas	6, 7, 9, 15
HackerRank SQL	Muitos exercícios por tema, bom para volume de prática	1 a 5
Documentação do PostgreSQL	Referência oficial, com capítulos sobre funções de janela, índices, `MERGE` e particionamento	Todos

Fonte	Para que usar	Módulos
Kaggle: Olist	Página da base, com notebooks de análises feitas por outras pessoas para comparar ideias (veja só depois de tentar)	Desafios